

# Ansible & Automation

Interview Questions & Answers

*Interview Preparation Guide*

Fundamentals - playbooks & roles - variables & templating - modules - network automation - IaC  
- AWS - CI/CD - reliability & scale - troubleshooting - security - AWX/Tower - senior-level design

## Table of Contents

---

|                                   |    |
|-----------------------------------|----|
| 1. Ansible Fundamentals           | 3  |
| 2. Playbooks & Roles              | 5  |
| 3. Variables & Templating         | 6  |
| 4. Modules                        | 7  |
| 5. Network Automation             | 8  |
| 6. Infrastructure as Code         | 9  |
| 7. Cloud Automation (AWS)         | 10 |
| 8. CI/CD & DevOps                 | 11 |
| 9. Reliability & Scale            | 12 |
| 10. Troubleshooting               | 13 |
| 11. Security                      | 14 |
| 12. AWX / Ansible Tower           | 14 |
| 13. Senior-Level Design Questions | 15 |

## How to Use This Guide

---

This guide answers common interview questions on Ansible and infrastructure/network automation, grouped by theme. Each answer is concise and aimed at engineer-to-senior level: it states the core idea first, then the practical detail an interviewer wants to hear. Use the table of contents to jump to a category, and treat the senior-level design section as prompts to practice structured, trade-off-driven answers on a whiteboard.

### 1. Ansible Fundamentals

#### Q1. What problem does Ansible solve?

Ansible solves configuration management, application deployment, and orchestration by letting you describe desired state declaratively and apply it consistently across many hosts. It eliminates manual, error-prone, snowflake configuration by making infrastructure reproducible, version-controlled, and auditable. Its agentless, SSH/API-based model means there is nothing to install on managed nodes, lowering the barrier to automating servers, network devices, and cloud APIs.

#### Q2. How does Ansible work internally?

From a control node, Ansible reads an inventory and a playbook, connects to each target (SSH for Linux, WinRM for Windows, or device/cloud APIs), and executes tasks. For most modules it works like this:

- Gathers facts (unless disabled) to learn the host's current state.
- Ships the module code (usually Python) plus arguments to the target, executes it, and collects JSON results.
- The module enforces desired state and reports changed/ok/failed; Ansible removes the temporary module afterward.
- Network/cloud modules often run locally on the control node and talk to the device or cloud API instead of copying code to the target.

#### Q3. What is the difference between push and pull automation?

Push (Ansible's default) means the control node initiates connections and pushes configuration out to targets on demand - simple, agentless, and good for orchestration. Pull means each node periodically fetches and applies its own config from a central source (e.g., Puppet/Chef agents, or ansible-pull from cron). Pull scales well for huge fleets and self-healing, but adds an agent/scheduler; push gives precise, ordered, on-demand control which matters for sequenced network changes.

#### Q4. What are inventories and how are they structured?

An inventory is the list of managed hosts plus their grouping and variables. It can be static (INI/YAML files) or dynamic (a script/plugin that queries a source of truth like AWS, NetBox, or a CMDB). Typical structure:

- Hosts defined individually or via ranges/patterns.
- Groups of hosts, and groups-of-groups (children) for hierarchy.
- Variables at host or group level, ideally in `host_vars/` and `group_vars/` directories.
- Special groups: 'all' (every host) and 'ungrouped'.

### **Q5. What are groups and group variables?**

Groups categorize hosts by role, location, environment, or vendor (e.g., webserver, dc1, prod, cisco). Group variables apply to every host in a group, defined inline or in `group_vars/<group>.yml`, letting you set shared settings (ports, NTP servers, credentials) once. Hosts can belong to many groups, and you target plays/tasks by group patterns; `group_vars/all.yml` holds globally shared values.

### **Q6. Difference between playbook, role, collection, and module?**

These are increasing units of organization and reuse:

- Module: a single unit of work (e.g., copy, yum, ios\_config) that enforces one piece of state and returns JSON.
- Playbook: a YAML file mapping plays (host groups) to ordered tasks that call modules.
- Role: a reusable, self-contained bundle of tasks, handlers, templates, defaults, and vars with a standard directory layout.
- Collection: a distributable package (via Galaxy/Automation Hub) bundling modules, plugins, roles, and playbooks under a namespace (e.g., cisco.ios).

### **Q7. What is idempotency and why is it important?**

Idempotency means running the same playbook repeatedly produces the same end state and makes changes only when the actual state differs from desired state. It is important because it makes automation safe to re-run, enables drift correction, supports convergence, and gives accurate change reporting - critical for production network and infrastructure changes where unnecessary actions cause outages.

### **Q8. How does Ansible ensure idempotency?**

Well-written modules check current state before acting and only modify what is needed, reporting `changed=true` solely when a real change occurred. Ansible itself does not magically make tasks idempotent - it depends on the module. Declarative modules (copy, template, package, ios\_config) are idempotent; raw command/shell modules are not unless you add guards (creates/removes, changed\_when, when conditions, check mode).

### **Q9. What are facts and fact gathering?**

Facts are system properties Ansible discovers about a host (OS, IPs, memory, interfaces, etc.), exposed as `ansible_facts` variables. The setup module gathers them at the start of a play by default. Facts let playbooks make decisions (e.g., choose a package manager) and can be cached to speed up subsequent runs.

### **Q10. When would you disable fact gathering?**

Disable gathering (`gather_facts: false`) when you do not use facts and want speed - e.g., large fleets, simple network pushes, or targets where setup is slow/unsupported. It noticeably cuts run time at scale. If you need a few facts, gather selectively with the `gather_subset` option or run setup on demand rather than disabling entirely.

## 2. Playbooks & Roles

### Q11. How do you structure a large Ansible project?

Use a convention-driven layout that separates inventory, roles, and group/host variables, and keep logic in roles:

- Per-environment inventories (inventories/prod, inventories/dev) each with group\_vars/ and host\_vars/.
- Reusable roles/ for each function; thin top-level playbooks that just include roles.
- Collections/requirements (requirements.yml) pinned to versions.
- Shared defaults in group\_vars/all; secrets in Vault; everything in Git.

### Q12. How do roles improve maintainability?

Roles encapsulate related tasks, handlers, templates, files, and variables behind a stable interface, so you reuse them across playbooks and environments without copy/paste. They enforce a standard structure, make ownership and testing (Molecule) easier, and let teams compose complex automation from small, independently versioned building blocks.

### Q13. What is the purpose of defaults, vars, handlers, and templates directories?

Each role subdirectory has a defined role:

- defaults/: lowest-precedence variables meant to be overridden by users - good for tunables.
- vars/: higher-precedence, role-internal variables not meant to be overridden lightly.
- handlers/: tasks triggered by notify, run once at the end (e.g., restart service).
- templates/: Jinja2 templates rendered with the template module into config files.

### Q14. Difference between include\_tasks and import\_tasks?

import\_tasks is static: tasks are parsed at playbook parse time, so tags/handlers are known up front but you cannot use runtime variables to choose the file or loop over it. include\_tasks is dynamic: evaluated at runtime, so it supports variable file names, loops, and conditionals on the include itself, at the cost of less static visibility (tags must be applied carefully).

### Q15. Difference between include\_role and import\_role?

Same static-vs-dynamic distinction as tasks. import\_role is processed at parse time (role tasks become part of the play, handlers/defaults available early); include\_role is processed at runtime, allowing conditional/looped role inclusion and runtime variable selection. Use import\_ for predictable structure and dynamic include\_ when you must decide at runtime.

### Q16. When would you use handlers?

Use handlers for actions that should run only once and only if something changed - classically restarting/reloading a service after its config file is updated. Multiple tasks can notify the same handler; it runs a single time at the end of the play (or when flushed with meta: flush\_handlers). This avoids redundant restarts and keeps runs idempotent.

### Q17. How do you avoid duplicated code across playbooks?

Factor shared logic into roles and collections, parameterize them with variables/defaults, and reuse via include\_role/import\_role. Use group\_vars/host\_vars for environment differences instead of duplicating plays, keep common tasks in a shared role, and publish stable collections so multiple repos consume one source.

### 3. Variables & Templating

#### Q18. Explain variable precedence in Ansible.

Ansible merges variables from many sources with a defined precedence (low to high, abbreviated):

- role defaults (lowest)
- inventory file/group\_vars/all -> group\_vars/<group> -> host\_vars
- play vars, vars\_files, role vars
- task vars, block vars, registered vars, set\_fact
- extra vars passed with -e (highest, always wins).

#### Q19. How do you store environment-specific variables?

Keep separate inventories per environment, each with its own group\_vars/ and host\_vars/ (e.g., inventories/prod/group\_vars/all.yml). Put environment-invariant defaults in role defaults and only override the deltas per environment. This keeps prod/dev differences explicit, diffable in Git, and avoids branching logic inside playbooks.

#### Q20. Difference between vars\_files, host\_vars, and group\_vars?

All set variables but differ in scope and loading:

- group\_vars/<group>.yml: auto-loaded for every host in that group.
- host\_vars/<host>.yml: auto-loaded for one specific host (higher precedence than group\_vars).
- vars\_files: explicitly included within a play; loaded at play scope and useful for shared or Vault-encrypted files.

#### Q21. What is Jinja2 templating?

Jinja2 is the templating engine Ansible uses to render variables, expressions, conditionals, loops, and filters - both in templates/\*.j2 files (via the template module) and inline in playbooks ({{ var }}). It enables dynamic configuration generation, e.g., building a per-device router config from variables and loops, with filters (default, map, to\_yaml) for transformation.

#### Q22. How do you validate generated configurations before deployment?

Validate before applying to avoid pushing broken config:

- Use the template module's 'validate' option to run a syntax checker on the rendered file (e.g., visudo -cf, nginx -t).
- Render in check/diff mode (--check --diff) to preview changes.
- Lint with ansible-lint and yamllint; run dry-runs in a staging environment.
- For network devices, use platform validate/commit-confirm features and diff against a golden config.

#### Q23. How do you manage secrets securely?

Use Ansible Vault to encrypt secret files/variables at rest, integrate an external secrets manager (HashiCorp Vault, AWS Secrets Manager, CyberArk) via lookups for dynamic retrieval, and store credentials in AWX/Tower credential objects rather than in Git. Never commit plaintext secrets, restrict who can decrypt, and use no\_log to keep secrets out of output.

## 4. Modules

### Q24. Difference between command, shell, and raw modules?

All run commands but with key differences:

- **command**: runs a binary without a shell - safer, no pipes/redirects/env expansion; not idempotent by default.
- **shell**: runs through `/bin/sh`, so pipes, redirects, and variables work - more powerful but more injection-prone.
- **raw**: sends the command over the connection with no Python required on the target - used for bootstrapping (installing Python) or simple network devices.

### Q25. When would you use the uri module?

Use `uri` to interact with HTTP/REST APIs from a playbook - GET/POST/PUT/DELETE, send JSON bodies, set headers/auth, check status codes, and register responses. It is ideal for calling cloud or controller APIs (e.g., a load balancer, SDN controller, or webhook) and for health checks/validation steps, often with retries via `until/retries`.

### Q26. Difference between copy, template, and assemble modules?

Three ways to place files on a target:

- **copy**: transfers a static file (or inline content) unchanged, with ownership/perms.
- **template**: renders a Jinja2 `.j2` file with variables before transferring - for dynamic config.
- **assemble**: concatenates multiple fragment files from a directory into one file - handy for building configs from drop-in snippets.

### Q27. How does `ios_config` / `arista.eos.eos_config` / `junos_config` achieve idempotency?

These network config modules pull the running configuration, compare your intended lines/blocks against it, and push only the missing/differing commands - reporting changed only when the device config actually changes. They support context (parents), match/replace strategies, and backup, and many platforms add `commit/commit-confirm` and `diff` so changes are validated and reversible.

### Q28. How would you write a custom Ansible module?

Write a module (usually Python) that reads arguments, does the work idempotently, and returns JSON:

- Use `AnsibleModule` from `ansible.module_utils.basic` to parse `argument_spec` and support `check_mode`.
- Detect current state, change only if needed, and set changed accordingly.
- Return results with `module.exit_json(...)` or errors with `module.fail_json(...)`.
- Place it in `library/` or a collection's `plugins/modules/`, document it, and test with unit tests + `Molecule`.

### Q29. What are module return values?

Modules return a JSON dict that Ansible surfaces - standard keys include `changed`, `failed`, `msg`, `rc`, `stdout/stderr` (for commands), and module-specific data. You capture these with `register` and then branch with `when`, loop over results, or assert on them. Common derived facts include `results` (for loops) and `ansible_facts` (to inject new facts).

## 5. Network Automation

### Q30. How does Ansible connect to network devices?

Network modules typically run on the control node and connect to devices using a connection plugin set by `ansible_connection` - `network_cli` (SSH CLI), `netconf` (NETCONF/XML over SSH), or `httpapi` (REST/HTTP). You specify `ansible_network_os` (e.g., `cisco.ios.ios`), credentials, and `become/enable` as needed; the platform collection handles the device-specific dialect.

### Q31. Difference between `network_cli`, `netconf`, and `httpapi`?

Three connection plugins for network gear:

- `network_cli`: screen-scrapes the SSH CLI - broad device support, but parsing-based and less structured.
- `netconf`: structured XML config/state over SSH with candidate datastores and `commit/rollback` - transactional and model-driven.
- `httpapi`: talks to a device/controller REST API over HTTP(S) - used by platforms that expose APIs (e.g., some Arista/Cisco/F5).

### Q32. When would you choose NETCONF over CLI automation?

Choose NETCONF when you need structured, model-driven (YANG) configuration with transactional semantics - candidate config, validate, commit, and rollback - and reliable machine-readable state instead of fragile CLI scraping. It is preferable for complex, high-risk changes and multi-step transactions; CLI automation remains useful for older devices or simple, well-understood commands.

### Q33. How would you back up configurations before changes?

Use the platform config module's backup option (e.g., `ios_config: backup: yes`) or a dedicated `_facts/gather` to fetch the running config, then store it with a timestamp in Git or an artifact store before applying changes. Backups give you a known-good rollback target and an audit trail; automate them as the first task in every change play.

### Q34. How do you validate post-change state?

Verify the device behaves as intended after the change:

- Gather facts/operational state and assert on it (interface up, BGP neighbors established, route present).
- Use `ios_command/eos_command` with `wait_for/until` to poll until conditions are met.
- Diff running config against the golden/intended config.
- Run end-to-end probes (ping/uri/health checks) and roll back on failure.

### Q35. How would you automate BGP deployment across hundreds of routers?

Drive it from a source of truth with roles and safe rollout:

- Model BGP intent (ASN, neighbors, policies) as variables in `group_vars/host_vars` or a NetBox/IPAM source.
- Render config via templates/resource modules; validate in check/diff mode.
- Roll out with serial + canaries, backing up first and asserting BGP state after each batch.
- Use `commit-confirm/rollback` and `stop-on-failure` to limit blast radius.

**Q36. How do you manage multi-vendor environments?**

Abstract intent from vendor syntax: keep vendor-neutral variables (the desired state) and use per-vendor roles/collections (cisco.ios, arista.eos, junipernetworks.junos) or resource modules to render the right config. Group hosts by platform, set `ansible_network_os` accordingly, and where possible use standards (NETCONF/YANG, OpenConfig) to reduce divergence.

**Q37. How do you prevent configuration drift?**

Make the repo/source-of-truth authoritative and re-converge regularly: schedule idempotent playbooks (or AWX jobs) that compare and correct config, run compliance checks in check/diff mode to detect drift, alert on unexpected diffs, and forbid manual changes via process and RBAC. Drift becomes a reported diff that automation remediates.

## 6. Infrastructure as Code

**Q38. How does Ansible compare with Terraform?**

Terraform is declarative provisioning with explicit state - it creates/updates/destroys cloud resources and tracks them in a state file (great for immutable infrastructure). Ansible is procedural-leaning configuration management - excellent at configuring existing hosts, app deployment, and orchestration, and agentless. They overlap but are complementary: Terraform builds the infrastructure, Ansible configures what runs on it.

**Q39. When would you use Terraform instead of Ansible?**

Use Terraform when you need to provision and lifecycle-manage cloud infrastructure with dependency graphs and authoritative state (VPCs, subnets, managed services), where its plan/apply and drift detection on resources shine. It is better than Ansible for create/destroy of large cloud topologies and for treating infrastructure as immutable.

**Q40. Can Terraform and Ansible be used together?**

Yes - a common pattern is Terraform provisions resources (and can output an inventory), then Ansible configures the instances and deploys applications. You can call Ansible from a Terraform provisioner (sparingly), or better, run them as separate pipeline stages with Terraform outputs feeding Ansible's dynamic inventory.

**Q41. How do you maintain source-of-truth for infrastructure?**

Keep declarative definitions (Terraform code, Ansible vars/inventories) in Git as the single source of truth, with code review and CI. For dynamic data, integrate an authoritative system (NetBox/IPAM, CMDB, cloud APIs) via dynamic inventory so automation reads intent from one place. Avoid out-of-band manual edits; reconcile drift back to the SoT.

**Q42. How do you handle rollback of infrastructure changes?**

Plan rollback up front:

- Version everything in Git so you can revert code and re-apply a previous known-good state.
- For config changes, back up first and use `device commit-confirm` / `restore-from-backup`.
- For immutable infra, roll back by deploying the previous image/version (blue-green).
- For Terraform, revert code and apply, or restore prior state carefully; prefer forward-fix when destroy is risky.

**Q43. What challenges exist with mutable vs immutable infrastructure?**

Mutable infra is changed in place (patch/config), which is flexible but accumulates drift and snowflakes and complicates rollback. Immutable infra replaces whole artifacts (rebuild image, redeploy), giving consistency and easy rollback but needing robust build/deploy pipelines and handling of state/data. Networks are often inherently mutable, so idempotency and drift control matter more there.

## 7. Cloud Automation (AWS)

**Q44. How would you automate VPC creation?**

Use the `amazon.aws` collection (`amazon.aws.ec2_vpc_net`, `ec2_vpc_subnet`, `ec2_vpc_route_table`, `ec2_vpc_igw`, `ec2_vpc_nat_gateway`) driven by variables for CIDRs, AZs, and tiers, ideally within a role. Make it idempotent and tag everything; for large/standardized topologies many teams prefer Terraform for provisioning and Ansible for configuration, but Ansible can do both.

**Q45. How would you provision EC2 instances using Ansible?**

Use `amazon.aws.ec2_instance` with parameters for AMI, type, subnet, security groups, key, and tags; register output and optionally add hosts to an in-memory inventory with `add_host` for immediate configuration. Combine with `ec2_key`, `ec2_security_group`, and `wait_for` to ensure SSH is ready, then run configuration roles.

**Q46. How do Ansible dynamic inventories work in AWS?**

The `aws_ec2` inventory plugin queries EC2 (and other services) at runtime and builds hosts/groups from tags, regions, VPCs, and attributes using `compose/keyed_groups`. This removes static host lists - instances are discovered live and grouped (e.g., by `tag:Role`) so playbooks always target the current fleet. Credentials come from standard AWS auth (`env`, `profile`, `instance role`).

**Q47. How do you manage IAM roles and policies with Ansible?**

Use `community.aws/amazon.aws` IAM modules (`iam_role`, `iam_policy`, `iam_managed_policy`, `iam_user`) with policy documents (often rendered from Jinja2/JSON) kept in Git. Apply least privilege, prefer managed policies and roles over inline user keys, and make the tasks idempotent so policies converge to the declared state.

**Q48. How would you automate security group management?**

Define security groups declaratively with `amazon.aws.ec2_security_group`, listing intended ingress/egress rules from variables - the module reconciles actual rules to match (idempotent). Reference group IDs/names for east-west rules, source rules from a source of truth, and review changes in CI to prevent accidental 0.0.0.0/0 exposure.

**Q49. How would you deploy to multiple AWS accounts?**

Use per-account credentials/roles and `assume-role` (`sts_assume_role` or `profiles`), drive `account/region` as variables, and loop or run per-account jobs. In AWX, model each account as a credential and use job templates; structure inventories/vars per account and enforce guardrails (SCPs, naming, tagging) consistently across accounts.

### **Q50. How do you handle secrets in AWS?**

Store secrets in AWS Secrets Manager or SSM Parameter Store and fetch them at runtime with the corresponding lookup plugins, or use IAM instance roles so no static keys are needed. Encrypt anything at rest with Vault if it must live in Git, use `no_log`, and rotate via the secrets manager rather than editing playbooks.

## **8. CI/CD & DevOps**

### **Q51. How do you integrate Ansible into Jenkins?**

Run Ansible from a Jenkins pipeline stage (declarative Jenkinsfile) using the Ansible plugin or a plain sh step, injecting credentials via the Credentials store/Vault and passing inventory and extra-vars. Gate with lint/syntax/Molecule stages, require approvals for prod, and archive logs/artifacts. Many teams trigger AWX/Tower job templates from Jenkins instead of running ansible directly.

### **Q52. What should happen in an Ansible CI/CD pipeline?**

A typical pipeline progresses through quality gates:

- Lint/syntax: `yamllint`, `ansible-lint`, `ansible-playbook --syntax-check`.
- Unit/role tests: Molecule converge + idempotence + verify in ephemeral environments.
- Dry run: `--check --diff` against staging.
- Approval gate, then controlled apply to prod (serial/canary), with post-checks and notifications.

### **Q53. How do you test playbooks before production?**

Use Molecule to spin up containers/VMs and run converge, idempotence, and verify scenarios; run `--check --diff` and `--syntax-check`; and validate against a staging environment that mirrors prod. For network devices, use virtual labs (containerlab, vrnetlab, EVE-NG) or vendor simulators before touching real hardware.

### **Q54. What is Molecule?**

Molecule is the standard framework for developing and testing Ansible roles. It manages test instances (Docker, Podman, cloud, etc.), runs lifecycle stages (create, converge, idempotence, verify, destroy), and integrates linting and assertion frameworks (Ansible asserts, testinfra). It catches non-idempotent tasks and regressions early in CI.

### **Q55. How do you implement GitOps with Ansible?**

Make Git the single source of truth: changes to inventories/vars/playbooks go through pull requests and review, CI validates them, and a controller (AWX/Tower, or a pipeline) applies the merged state automatically. Reconcile regularly so the live environment converges to what is in Git, with full audit history and the ability to revert by reverting commits.

### **Q56. How do you perform peer reviews for automation code?**

Treat automation as code: require PRs with at least one reviewer, run CI (lint, Molecule, `--check`) as merge gates, and review for idempotency, secrets handling, blast radius (serial/limits), and clear variables/defaults. Use CODEOWNERS for sensitive roles and keep changes small and well-described.

**Q57. How do you manage versioning of Ansible collections?**

Pin collection versions in requirements.yml and install with ansible-galaxy; follow semantic versioning and test upgrades in CI before bumping. Host internal collections in Automation Hub/a Galaxy server or Git, tag releases, and keep a changelog so consumers upgrade deliberately rather than tracking latest.

## 9. Reliability & Scale

**Q58. How does Ansible scale to thousands of devices?**

Scale with parallelism (forks), batching (serial), fact caching, and limiting gather/work to what is needed; for very large fleets use AWX/Tower with execution nodes or ansible-pull. Use dynamic inventory, mitogen or persistent connections where appropriate, and break work into targeted plays to keep runs fast and resilient.

**Q59. What is forks and how does it affect performance?**

forks sets how many hosts Ansible configures in parallel (default 5). Increasing it speeds up large runs by processing more hosts simultaneously, but is bounded by control-node CPU/memory/file descriptors and target/API rate limits. Tune it (e.g., 25-100) based on capacity; too high can overwhelm the control node or downstream APIs.

**Q60. How do serial deployments work?**

The serial keyword limits how many hosts in a play run at once - e.g., serial: 1, then 10%, then the rest - so changes roll out in controlled batches. If a batch exceeds the failure threshold (max\_fail\_percentage), the play stops, containing blast radius. It is the primary mechanism for canary and rolling network changes.

**Q61. What is a rolling deployment?**

A rolling deployment updates the fleet in successive batches (via serial), keeping the service available by changing only a subset at a time and validating each batch before proceeding. Combined with load-balancer drain/health checks, it minimizes downtime and lets you halt on the first sign of trouble.

**Q62. How would you automate changes with minimal blast radius?**

Constrain scope and verify continuously:

- Canary first (serial: 1 or a small percent) and stop on failure (max\_fail\_percentage: 0).
- Back up config and use --check --diff before applying.
- Validate state after each batch and use commit-confirm/auto-rollback on network gear.
- Use --limit to target subsets and tags to run only relevant tasks.

**Q63. How would you recover from a failed automation run?**

Rely on idempotency to safely re-run after fixing the cause; use backups and commit-confirm/rollback to restore network devices, and revert code in Git for IaC. Use --limit @retry files to target only failed hosts, inspect logs/registered results to find the failure point, and design plays with clear, reversible steps and blocks/rescue for cleanup.

**Q64. How do you make automation resilient to transient failures?**

Add retries/until with delay for flaky operations (API calls, services coming up), use wait\_for/wait\_for\_connection to synchronize on readiness, set sensible timeouts, and use block/rescue/always for error handling and cleanup. Make tasks idempotent so retries are safe, and back off to avoid hammering rate-limited APIs.

## 10. Troubleshooting

### Q65. Why would a task show changed every run?

Usually the task is not idempotent - command/shell without guards (creates/removes, changed\_when), a template whose rendering differs each run (timestamps, unordered dicts), or a module that always reports changed. Fix by using declarative modules, adding changed\_when/creates, sorting/normalizing template inputs, or comparing state before acting.

### Q66. How do you troubleshoot variable precedence issues?

Print the effective value with debug (var=) and use ansible-inventory --host to see merged inventory vars; trace where it is set (group\_vars, host\_vars, role vars, extra-vars). Remember -e extra-vars always wins and role defaults always lose. Reduce surprises by minimizing overlapping definitions and documenting where each var lives.

### Q67. Why might an Ansible playbook be slow?

Common causes and fixes:

- Fact gathering on every host - disable or scope it, enable fact caching.
- Low forks - increase parallelism within control-node limits.
- New SSH connection per task - enable pipelining and persistent connections (ControlPersist).
- Serial loops over slow APIs - batch, use async, or add retries with backoff.

### Q68. What causes SSH connection failures?

Wrong credentials/keys, host key verification issues, incorrect ansible\_user/port/become settings, firewalls/security groups blocking 22, target not ready (boot/cloud-init), or missing Python. Debug with -vvvv, test manual ssh, check known\_hosts, and use wait\_for\_connection after provisioning.

### Q69. How do you debug Jinja template rendering issues?

Render and inspect with debug (msg="{{ ... }}") or the template module in --check --diff, watch for undefined variables (set them or use default()), and check filters/types. Use ansible-playbook -vvv to see errors, and break complex expressions into intermediate set\_fact steps to isolate the problem.

### Q70. How do you troubleshoot network\_cli connectivity problems?

Work from connection to command:

- Confirm ansible\_connection=network\_cli, correct ansible\_network\_os, and credentials/become (enable).
- Run with -vvvv to see the SSH handshake and prompts; verify reachability and SSH access manually.
- Check the persistent connection settings/timeouts and that the device prompt is recognized.
- Validate the right platform collection is installed and matches the OS.

## 11. Security

### Q71. What is Ansible Vault?

Ansible Vault encrypts sensitive data (variables or entire files) at rest using a password or key, so secrets can live in Git safely. You create/edit with `ansible-vault` and supply the password at runtime (`--ask-vault-pass`, a password file, or a vault id). You can encrypt individual strings or whole files and combine multiple vault ids for different scopes.

### Q72. How do you rotate secrets?

Prefer an external secrets manager (HashiCorp Vault, AWS Secrets Manager) so rotation happens centrally and playbooks fetch current values via lookups. For Vault-encrypted files, use `ansible-vault rekey` to change the vault password and update the stored secret, then re-run automation. Automate rotation on a schedule and avoid hardcoding.

### Q73. How do you prevent secrets leaking in logs?

Set `no_log: true` on tasks that handle secrets, avoid echoing them via debug/command output, and use modules that mask sensitive return values. In CI/AWX, mask credentials and restrict log access; never pass secrets on the command line where they appear in process lists or history.

### Q74. How do you implement RBAC in AWX/Tower?

Use AWX organizations, teams, and roles to grant least-privilege access to inventories, projects, job templates, and credentials. Assign users to teams mapped from your SSO/LDAP, scope what each team can run or edit, and keep credentials in AWX (users launch jobs without ever seeing secrets). Audit who can launch which templates.

### Q75. How would you audit infrastructure changes?

Keep all changes in Git with PR history (who/what/why), capture AWX/Tower job logs and activity stream (who ran what, when, against which hosts), and store config backups/diffs per change. Forward logs to a SIEM, tag changes to tickets, and run periodic compliance scans to prove state matches intent.

## 12. AWX / Ansible Tower

### Q76. What advantages does AWX provide over CLI Ansible?

AWX (the upstream of Ansible Tower/Controller) adds an enterprise control layer:

- Web UI/API, RBAC, and SSO for controlled, multi-team access.
- Centralized, encrypted credential storage so users never see secrets.
- Job templates, scheduling, surveys, and a REST API for self-service and integration.
- Logging/audit (activity stream), notifications, and scalable execution nodes.

### Q77. How do inventories work in AWX?

AWX stores inventories as objects you can populate manually or via inventory sources (dynamic plugins like `aws_ec2`, or from your SCM project). It can sync on a schedule, apply smart inventories (filtered host sets), and attach credentials so jobs target the right, up-to-date hosts without static files.

**Q78. What are job templates?**

A job template is a reusable definition that ties together a project (playbook), an inventory, credentials, and options (limits, tags, verbosity, extra-vars). Users launch jobs from templates with consistent settings and proper RBAC; templates can be chained into workflows for multi-step automation.

**Q79. What are surveys?**

Surveys add a parameter form to a job template, prompting the user for inputs (which become extra-vars) with types, defaults, and validation. They enable safe self-service - e.g., choose environment or host - without editing playbooks, while constraining what users can supply.

**Q80. How do credentials work?**

AWX stores credentials (SSH keys, passwords, cloud keys, vault passwords, API tokens) encrypted, and injects them into jobs at runtime as environment variables or connection settings. Users with permission to launch a template use the credential without being able to read it, and credential types are pluggable for many systems.

**Q81. How would you integrate AWX with Git?**

Create a Project pointing at your Git repo (with SCM credentials), and AWX syncs the playbooks/roles on demand or on a schedule/webhook. Job templates then run the synced content, so Git remains the source of truth and GitOps-style workflows (PR -> merge -> sync -> run) are straightforward.

**Q82. How would you expose automation through self-service portals?**

Use job templates plus surveys behind RBAC so non-experts can launch vetted automation safely, or drive AWX's REST API from a custom portal/ServiceNow to trigger templates and return status. Constrain inputs via surveys, gate sensitive actions with approvals in workflows, and report results back to the requester.

## 13. Senior-Level Design Questions

**Q83. Design an automation platform for 10,000 network devices.**

Build a source-of-truth driven, scalable, safe platform:

- SoT: NetBox/IPAM + Git for intended state; dynamic inventory generated from it.
- Execution: AWX/Controller with multiple execution nodes (and/or mesh) for horizontal scale; tuned forks and fact caching.
- Workflow: render -> validate (check/diff) -> canary/serial rollout -> post-validate -> rollback on failure.
- Multi-vendor abstraction via platform collections; full logging/audit, RBAC, metrics, and scheduled drift remediation.

**Q84. How would you build a source-of-truth driven automation framework?**

Make an authoritative system (NetBox/CMDB + Git) hold intended state, generate dynamic inventory and variables from it, and have automation render and enforce config from that data only. Changes happen by editing the SoT (via PR), CI validates, and the controller converges devices; drift is detected by diffing live state against the SoT and remediated automatically.

**Q85. How would you implement configuration compliance at scale?**

Define compliance as code (intended config/policies), run scheduled idempotent checks in check/diff mode (or tools like Batfish/validation modules) across the fleet, and report pass/fail with diffs to a dashboard/SIEM. Auto-remediate low-risk findings, raise tickets for the rest, and track compliance as a measurable metric over time.

**Q86. How would you detect and remediate configuration drift?**

Continuously gather running config/state and diff it against the source of truth; alert on differences and trigger an idempotent remediation play (with approvals for risky changes) that re-converges the device. Combine scheduled runs, event-driven triggers, and strict RBAC/process to prevent out-of-band edits in the first place.

**Q87. How would you automate device onboarding end-to-end?**

Zero-touch from rack to managed:

- ZTP/DHCP delivers a base image/config so the device is reachable.
- Register it in the SoT (NetBox) with role/site/vars; dynamic inventory picks it up.
- Run a bootstrap role (credentials, management, hardening) then role-specific config.
- Validate state, add to monitoring/backup, and mark onboarded - all logged/audited.

**Q88. How would you safely deploy a BGP policy change to 500 routers?**

Treat it as a high-risk, staged change:

- Peer review the rendered diff; back up configs first.
- Canary on 1-2 routers, validate BGP sessions/routes and traffic, then expand in serial batches.
- Use commit-confirm/auto-rollback and stop-on-failure (max\_fail\_percentage: 0).
- Post-validate at each stage; have a tested rollback and a change/approval window.

**Q89. How would you design an approval workflow for production network changes?**

Use a workflow (AWX workflow templates or a CI pipeline) that runs validation and a dry-run diff, then pauses for human approval before the apply stage, with RBAC limiting who can approve and run. Tie it to change management (ticket reference), require successful pre-checks, and capture approver, diff, and results in the audit log.

**Q90. How would you build a multi-cloud infrastructure automation platform?**

Standardize on Git-based IaC with a common pipeline, use Terraform for cross-cloud provisioning and Ansible for configuration/app deployment, and abstract cloud differences behind modules/roles and per-cloud credentials. Centralize state, secrets, policy-as-code (OPA/Sentinel), and observability so each cloud is a pluggable backend under one workflow and governance model.

**Q91. How would you measure the success of an automation program?**

Track outcome and adoption metrics, not just activity:

- Lead time for changes and deployment frequency (DORA).
- Change failure rate and MTTR; number of incidents from manual error.
- Percentage of changes automated / devices under management / compliance score.
- Toil hours saved and drift incidents detected/remediated automatically.

**Q92. What are the biggest risks when scaling infrastructure automation?**

Automation multiplies both good and bad changes:

- Blast radius: one bad play can break thousands of devices - mitigate with canary/serial, check mode, and rollback.
- Drift and out-of-band changes undermining the source of truth.
- Secret sprawl/leakage and over-broad credentials - enforce Vault/RBAC/least privilege.
- Insufficient testing, brittle CLI scraping, and lack of observability/audit.